ChatGPT

Project Sensy – MVP to Launch Plan

Executive Summary

Project Sensy ka goal hai ek **premium minimalist UI** ke saath MVP launch karna, minimum cost pe. Is plan me har phase ke liye tasks, time estimates, deliverables, priorities, dependencies aur risk mitigation diye gaye hain. Supabase (Auth, Postgres DB, Storage, Edge Functions, Webhooks) ko backend ke liye use kiya jayega【5†L133-L135】【46†L99-L107】, frontend ke liye Next.js/Tailwind (shadcn/ui) with Framer Motion. Hosting/Vercel pe (preview/prod pipelines) setup karenge【9†L283-L292】【9†L307-L312】. Payments ke liye Razorpay/PhonePe, emails ke liye Resend (free 3000/mo【50†L24-L32】) istemal kareinge. WhatsApp messaging ke liye Meta Business API use karke opt-in flows, templates, webhooks implement honge【23†L178-L185】【23†L197-L205】. Google Apps Script se Sheets/Gmail automation triggers bhi banenge (installable triggers se mail actions)【31†L1393-L1404】【31†L1420-L1424】. Har release ke liye CI/CD (GitHub Actions + Vercel auto-deploy) aur testing (unit, E2E) strategy rakhi jayegi. Monitoring aur logging Vercel/Supabase tools se, aur zarurat padi to Sentry add kar sakte hain. Instant rollback ke liye Vercel feature use hoga【36†L206-L215】. Sabse pehle kuch core features (landing page, auth, payment, dashboard, admin) develop karenge, phir advanced modules (AI, workflows) add karenge. Free-tier limits cross hone par hi upgrade plan sochenge. Is plan me har phase me required resources (time, cost) aur risk mitigation bhi list hai.

Phase-by-Phase Plan

- **Phase 1: Design & Setup (2–3 din)** – *Priority: High*
- **Design:** Landing page aur dashboard ke wireframes banayen. Color palette aur typography finalize karein. (Colors: e.g. black `#000`, off-black `#181818`, white `#FFF`, accent `#5B8CFF`). Typography: Inter ya Geist (sans serif) with clear hierarchy. Breakpoints: mobile-first (Tailwind: sm:640px, md:768px, lg:1024px etc). Accessibility: ensure WCAG AA (contrast ≥4.5:1【34†L209-L218】). Minimal animations (Framer Motion for fades/slides).
- **Tech Setup:** Vercel me project create karke GitHub link karein. Supabase project setup karein (database, auth, storage). Initialize Next.js app with Tailwind. (Deliverables: Design mockups, project repos set up, basic Next.js scaffold).

- **Dependencies:** Domain registration, initial Git repo, design files.

- **Phase 2: Core Development (1.5–2 haftay)** – *Priority: Critical*

- **Authentication:** Supabase Auth configure karein. Email/Magic Link aur social login (Google, GitHub) enable karein【5†L133-L135】【5†L169-L177】. (Login, signup, forgot-password flows). Ensure JWT and RLS policies for user data. Deliver: Working signup/login pages.
- **Database Schema:** Create core tables (users, products, purchases/orders, plans, coupons). Use Postgres relations and RLS. Eg: `users(id PK, email, created_at)`, `orders(id, user_id FK, product_id, status, created_at)`. (See example below). Handle timestamps, indexes. Deliver: SQL migration scripts; sample data seed.
- **File Storage:** Supabase Storage bucket(s) for user uploads (images/files). Set bucket to private and use RLS policies for access【46†L101-L110】. Deliver: Configured buckets, test upload APIs.

- **Frontend Pages:** Build main pages – Landing Hero, Auth (Sign in/up), Dashboard (protected). Use Tailwind + shadcn components. Implement Dark/light mode toggle if needed. Deliver: Responsive layouts.

- **Dependencies:** Design assets, Supabase project, domain info (for env config).

- **Phase 3: Payments & Access (5–7 din)** – *Priority: High*

- **Payment Gateway:** Integrate Razorpay/PhonePe. Add checkout on frontend, create payment intent via API, handle success/failure callbacks. (Razorpay payments can call Vercel API or Edge function with webhook).
- **Order Flow:** On payment success webhook, use a Supabase Edge Function to process order: write `orders` row, update user access (unlock product). Deliver: Automated payment→order→access flow.
- **Emails:** Use Resend for transactional emails (OTP, purchase receipts). In Edge Function, after order success, call Resend API to send email (Resend free upto 3000/mo【50†L24-L32】). Add welcome email on signup. Deliver: Email templates in React Email.
- **Supabase Edge Functions:** Deploy a function (TypeScript/Deno) for payment webhook and any other server-side logic. Edge functions run globally【1†L127-L134】 and can call Supabase client. (Store secrets via Supabase environment variables).
- **Admin Panel Setup:** Stub admin pages (later detail in Phase 4). For now, focus on user-facing flows.

- **Dependencies:** Access to payment gateway sandbox keys, Resend API key.

- **Phase 4: Admin & Additional Features (1–2 haftay)** – *Priority: Medium*

- **Admin Panel:** Build React pages (protected by admin RLS or a separate Supabase Auth `role=admin`). Features: View users, orders, products, manage plans/coupons. Use Supabase Auth API to get user list. Deliver: CRUD admin UI.
- **AI & Automation (Basic):** Placeholder for AI chat/workflow if included; else scaffold for future. Google Apps Script: set up an Apps Script project to automate (e.g. daily reporting). Use triggers (time-based) to fetch data from Supabase (via REST) or Google Sheets. Note: simple triggers can't call external APIs; use installable/time-driven triggers【31†L1367-L1375】【31†L1420-L1424】. Deliver: example GAS script (e.g. daily summary email).
- **Meta/WhatsApp Integration:** Register WhatsApp Business Account, get API token, verify number. Create message templates (authentication, utility) in Facebook Business Manager. Implement basic flow: on certain DB event (e.g. order shipped), call Edge Function that hits WhatsApp Cloud API to send message. (Use Cloud API "messages" endpoint with template name). Deliver: Working API call (test in sandbox).
- **CI/CD:** Configure GitHub Actions: on PR run ESLint, unit tests. On merge to main, Vercel auto-deploy【9†L307-L312】. (Add `vercel.json` for redirects if needed.)

- **Dependencies:** Completed core features; all API keys (WhatsApp, payment).

- **Phase 5: Testing & Launch (3–5 din)** – *Priority: Critical*

- **Testing:** Write Unit tests (Jest/React Testing Library) for critical components and utils. Set up E2E tests (Cypress/Playwright) for user flows (signup, payment, dashboard). Add tests to CI.
- **Performance & Accessibility:** Check Lighthouse scores, fix issues. Ensure color contrast ≥4.5:1【34†L209-L218】, keyboard nav, aria labels.

- **Monitoring/Logging:** Integrate Sentry (edge functions, frontend) or use Supabase logs. Set up alerts for errors or high latencies. Ensure Vercel logs accessible.
- **Rollback Plan:** Define rollback steps (Vercel Instant Rollback【36†L206-L215】, DB backups). Document how to revert Git deployments if needed.

- **Deployment:** Final merge to main, verify production domain, SSL. Run smoke tests (sanity checks). (Deliverables: live MVP at production URL).

- **Phase 6: Post-Launch Enhancements** – (Ongoing)

- Collect user feedback; iterate UI/UX.
- Add advanced features: AI chat, analytics dashboards, team workspaces, WhatsApp marketing campaigns (broadcasts).
- Scale Supabase (pro plan) if usage grows; consider caching or read replicas.

<table> <thead><tr><th>Phase</th><th>Time Estimate</th><th>Priority</th><th>Key Deliverables</th></tr></thead> <tbody> <tr><td>1. Design & Setup</td><td>~3 days</td><td>High</td><td>Wireframes, color scheme, Next.js/Supabase scaffold</td></tr> <tr><td>2. Core Dev (Auth + DB)</td><td>~10 days</td><td>Critical</td><td>Auth, DB schema, login/signup pages</td></tr> <tr><td>3. Payments & Email</td><td>~7 days</td><td>High</td><td>Payment integration, order flow, email notifications</td></tr> <tr><td>4. Admin & Integrations</td><td>~10 days</td><td>Medium</td><td>Admin panel, WhatsApp messages, GAS automation</td></tr> <tr><td>5. Testing & Launch</td><td>~5 days</td><td>Critical</td><td>Test suites, monitoring, production deploy</td></tr> </tbody> </table>

Supabase Implementation Steps

- **Authentication:** Use Supabase Auth for user accounts【5†L133-L135】. Enable email login with magic link or OTP, and OAuth (Google/GitHub)【5†L169-L177】. In Supabase dashboard, configure redirect URLs and email templates. Under the hood, Supabase uses JWT and stores users in a special schema【5†L144-L153】. Use Supabase Client in Next.js to handle sign-in/sign-out. Apply Row-Level Security (RLS) on tables to limit data access per user (e.g. `profile.user_id = auth.uid()`).

- **Database Schema:** Design tables and relationships. Example:

```
create table profiles (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users(id),
  email text unique not null,
  name text,
  created_at timestamptz default now()
);

create table products (
  id serial primary key,
  name text not null,
  price int not null
);
```

```
create table orders (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users(id),
  product_id int references products(id),
  status text not null,
  created_at timestamptz default now()
);

-- Example RLS policy: only allow users to see their own orders
create policy "Users can read own orders" on orders
  for select using (user_id = auth.uid());
```

Supabase is PostgreSQL, so you can use SQL migrations or the Dashboard editor. Keep tables lean initially. For exports/logs, a separate analytics bucket can be used.

- **Storage:** Use Supabase Storage for user-uploaded files. Create a bucket (e.g. "user-content"). By default buckets are private – use RLS to allow users to upload/download their own files 【46†L101-L110】. Example:

  ```
  -- Allow authenticated users to upload to their own path
  create policy "Allow upload if logged in" on storage.objects
    for insert using (
      auth.role() = 'authenticated'
    );
  ```

When serving images, Supabase provides a CDN and on-the-fly image transformation【46†L101-L110】.

- **Edge Functions:** For server-side logic and webhooks, use Supabase Edge Functions【1†L127-L134】. Edge functions are TypeScript (Deno) and run close to users. Example uses: handle Stripe/Razorpay webhooks, send emails, call WhatsApp API. Use `supabase.functions.invoke()` or direct HTTP. Store secrets in Supabase Dashboard (Project > Settings > API > Service Role key and custom secrets).

- **Database Webhooks:** Supabase supports DB Webhooks to call an external URL on table changes【3†L162-L170】. In the dashboard, create a new Webhook on the `orders` table for INSERT/UPDATE events. Internally it uses `supabase_functions.http_request`. For example:

  ```
  create trigger my_webhook after insert on public.orders for each row
    execute function supabase_functions.http_request(
      'https://your-backend.com/api/orders',
      'POST',
      '{"Content-Type":"application/json"}',
      '{"order_id": new.id, "user_id": new.user_id}',
      '1000'
    );
  ```

This sends a JSON payload to the given endpoint【3†L189-L197】. You can also call Edge Functions directly. Use these webhooks for real-time sync to other systems (e.g. Google Sheets) if needed.

- **Supabase Client:** In Next.js, use `@supabase/supabase-js` for frontend calls. Configure environment variables (`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`). For server-side calls (e.g. in API routes or Edge Functions), use the service role key (secure). Supabase also provides REST and GraphQL APIs for database access.

Vercel Deployment & CI/CD

- **Environments:** Vercel offers Local (dev), Preview, and Production environments【9†L283-L292】【9†L307-L312】. Link your GitHub repo to Vercel. Pushes to feature branches or PRs auto-trigger **Preview** deployments【9†L283-L292】, each with a unique URL for QA. Merge to `main` (production branch) auto-deploys **Production** (or use `vercel --prod`)【9†L307-L312】. Store secrets as Vercel Environment Variables: e.g. `SUPABASE_URL`, `SERVICE_ROLE_KEY`, `RESEND_API_KEY`, `WHATSAPP_TOKEN`. Pull variables locally with `vercel env pull`.

- **Deployment Pipeline:** Use GitHub Actions for CI: on every PR run `npm run lint`, `npm test`, and build. If green, Vercel deploys preview. After review, merge to main and let Vercel publish to prod. Use Vercel's Git integrations (GitHub App) for automatic comments with preview URLs. For staging or custom environments (Pro/Enterprise plan), you can set branch rules.

- **Rollback Plan:** Enable Vercel's **Instant Rollback** feature【36†L206-L215】. If a production deploy breaks, click "Instant Rollback" in Vercel dashboard to revert to a previous deploy. Note: rollback also reverts environment variables and configurations to that state【36†L211-L220】. For Hobby teams only the last deploy is available. After rollback, you must re-enable auto-deploy if changed【36†L274-L280】.

- **Testing & Checks:** Use Vercel Preview for final QA. Add Lighthouse or Deploy Checks if needed. Inspect logs via Vercel Dashboard or CLI. Use automatic Github checks to enforce code quality before merging.

Meta Business API (WhatsApp) Integration

- **Setup:** Register a WhatsApp Business Account (WABA) in Meta Business Manager. Complete business verification. On WhatsApp Cloud API, get a *Phone Number ID* and permanent *Access Token*. Use Meta's Cloud API endpoints (`https://graph.facebook.com/v17.0/<PHONE_NUMBER_ID>/messages`) to send messages.

- **Opt-in & Templates:** Only message users who have opted in【23†L178-L185】. Collect consent (e.g. checkbox). Any message outside 24-hour user-initiated window must use a pre-approved Template【23†L197-L205】 (e.g. order updates, OTP). Create templates (Utility/Authentication type) in Business Manager and submit for approval. Store template names in config.

- **Messaging Flows:**

- **Session Message (within 24h):** You can send free-form replies (text, media) via Cloud API after user message. Example: calling `/messages` with `type: "text"`.

- **Template Message (outside 24h):** Use templates. Example payload:

```
POST https://graph.facebook.com/v17.0/<PHONE_ID>/messages
{
  "messaging_product": "whatsapp",
  "to": "91999xxxxxxx",
  "type": "template",
  "template": {
    "name": "order_update",
    "language": { "code": "en_US" },
    "components": [
      {"type":"body","parameters":[{"type":"text","text":"Your order
#1234 has shipped."}]}
    ]
  }
}
```

(Not cited – example for reference.)

- **Rate Limits:** Meta enforces a *pair rate limit*: ~1 message/6s to the same user (≈600/h) 【26†L134-L142】. If exceeded, API returns an error. Also there are limits on number of contacts/chats (depends on business tier).

- **Webhooks (Inbound):** Set up a webhook URL (Vercel API or Supabase Function) to receive incoming message events and status updates. Subscribe to the "messages" and "message_status" fields in your app's webhook configuration. When the user sends a message or template response, you get a POST JSON which you can process (e.g. log into Supabase or trigger business logic).

- **Compliance:** Follow [WhatsApp Messaging Policy]【23†L178-L185】【23†L197-L205】. Outside 24h window, send only templates. Do not spam; respect blocks/opt-outs. Use escalation (e.g. email, phone) for customer service if automated replies fail【23†L209-L217】.

Google Apps Script Automation

- **Use Cases:** Use Apps Script for light automations, e.g. scheduled reports, syncing data with Google Sheets, or sending summary emails via Gmail. For example, a daily cron job can fetch latest orders from Supabase (via `UrlFetchApp`) and append to a Google Sheet. Or send a daily email of sales.

- **Triggers:** Apps Script offers *simple* and *installable* triggers【31†L1393-L1404】【31†L1420-L1424】. Simple triggers (`onOpen`, `onEdit`, `doGet`, `doPost`) run on container events but cannot call services requiring auth. For example, *simple* `onEdit(e)` cannot send Gmail (needs authorization)【31†L1420-L1424】. To send emails or access external APIs, use an **installable** trigger (e.g. time-driven trigger set in editor). Installable triggers run with authorization and longer execution.

- **Example:** To email a sheet summary every morning at 8am:

```
function dailySummary() {
  // Fetch data from Supabase (via REST or SDK if advanced)
  let response = UrlFetchApp.fetch('https://YOUR_SUPABASE_URL/rest/v1/
orders', {
    headers: { 'apikey': 'SUPA_KEY', 'Authorization': 'Bearer
SUPA_KEY' }
  });
  let orders = JSON.parse(response.getContentText());
  // Compose email via GmailApp
  let body = 'Today\'s orders: ' + orders.length;
  GmailApp.sendEmail('me@example.com', 'Daily Orders Summary', body);
}
```

Then in Apps Script UI, create a time-driven trigger for `dailySummary`.

UI/UX Specifications (Premium Minimalist)

- **Typography:** Use a clean sans-serif (e.g. **Inter**). Base font-size ~16px. Heading sizes scale (e.g. H1 2rem, H2 1.5rem). Maintain a 1.4x line-height. Ensure `:focus` outlines on interactive elements.

- **Color Tokens:**

- **Dark Mode (default):** Background `#000000` or `#181818`, card background `#1A1A1A`. Text `#FFFFFF` for high contrast. Accent color for buttons/links: `#5B8CFF`.
- **Light Mode (optional):** BG `#FFFFFF`, text `#000000`, accent same.

- *Ensure contrast ≥4.5:1 between text and background【34†L209-L218】.*

- **Components:** Reusable UI pieces (Tailwind + shadcn):

- **Buttons:** Primary (accent background, white text), Secondary (border/ghost style). Hover/active states with subtle color shift.
- **Cards:** Rounded corners (e.g. `rounded-3xl`), slight drop shadow, border outline (`border border-white/10`). No glass effects. Dark BG (#181818). Padding inside.
- **Inputs/Forms:** Underline or box style, large clear labels.
- **Navbar/Sidebar:** Collapsible sidebar with icons (Lucide icons), or top navbar for landing. Dark background, light text.
- **Tables:** Striped or clean layout for invoices/orders, horizontally scrollable on mobile.
- **Modals/Dialogs:** Centered with fade-in (Framer Motion). No heavy animations.

- **Icons:** Use a consistent set (e.g. *Lucide* or *Heroicons*).

- **Layout & Responsiveness:**

- Mobile-first grid: single-column layout on small screens, multi-col on desktop (e.g. 2-3 columns for dashboard cards).
- Tailwind breakpoints: `sm`, `md`, `lg`, `xl`. Ensure menu collapses on mobile, sidebar toggles.

- Utility spacing (padding/margin) in steps (4,8,16,32 px). Whitespace ample for premium feel.

- **Animations:** Subtle and purposeful. E.g.: Page transitions fade, button hover scale (Framer "whileHover"), modal fade/slide. Avoid jank.

- **Accessibility:** Alt text on images, ARIA labels on form fields, keyboard focus styles (use outline ring). Use semantic HTML (e.g. `<header>`, `<main>`, `<button>`). Ensure link focus is visible. Maintain 4.5:1 contrast【34†L209-L218】.

Example Component List & Wireframes

- **Landing Page:** Hero section with headline & CTA, features section (icons/text), screenshot gallery, pricing table, FAQ, footer.
- **Auth Pages:** Clean forms for login/signup with minimal text.
- **Dashboard:** Sidebar (Dashboard, Projects, Automation, AI, Billing, Settings). Topnav with user menu. Cards overview.
- **Projects Page:** List or grid of projects, each in a card with progress.
- **Automation/AI Pages:** Simple interfaces for triggers/bots (placeholder MVP).
- **Billing Page:** List of purchased products/plans, payment history.
- **Settings:** Profile settings and team (if future) options.
- **Admin Panel:** (separate route) tables for Users, Plans, Orders, Tickets.

(Wireframe diagrams omitted for brevity – focus on clean layout.)

CI/CD, Testing & Monitoring

- **CI/CD Pipeline:**
- **Code Repo:** Use GitHub. Protect `main` branch (required PRs).
- **Build & Test:** GitHub Actions runs on PR/commit. Steps: `npm install`, `npm run lint`, `npm test`, `npm run build`. Fail fast on errors.

- **Deployment:** Vercel auto-deploys previews for branches【9†L283-L292】; production on `main`. Use `vercel.json` for any redirects or rewrites.

- **Testing Strategy:**

- **Unit Tests:** Jest/React Testing Library for frontend components (e.g. login form, dashboard widgets). Supabase Edge Functions tests can use plain JS tests (Deno testing).
- **Integration Tests:** Test API routes (e.g. authentication, purchase endpoints) using Supertest or similar.

- **E2E Tests:** Use Cypress or Playwright to simulate user flows: Signup → Login → Purchase → Access content. Also test admin flows. Run these in CI on a stable preview URL (if possible).

- **Monitoring & Logging:**

- **Error Monitoring:** Integrate Sentry or LogRocket for frontend/runtime errors. For Edge Functions, use built-in logging (`console.log`) which shows in Supabase Logs.
- **Performance:** Use Vercel's built-in analytics and Google Analytics for traffic. Track important metrics (load time, API latency).

- **Logging:** Centralize logs: Vercel Logs for deployments, Supabase Dashboard for DB errors. Consider pushing critical events to an external log service if needed.

- **Rollback Plan:** As covered, use Vercel Instant Rollback【36†L206-L215】. For DB changes, do migrations with migration tool (e.g. `supabase migration`). Keep DB backups (Supabase free tier has 7-day retention on Pro).

Deployment & Release Checklist

1. **Pre-Release:**

2. [ ] Merge all PRs to `main` after code review.

3. [ ] Ensure all tests pass (unit + E2E).
4. [ ] Update version number (if any) and changelog.
5. [ ] Set `NODE_ENV=production` env vars.

6. [ ] Disable any debug flags.

7. **Deployment:**

8. [ ] Trigger production deploy on Vercel (auto via merge).

9. [ ] Verify environment variables on Vercel match staging config.

10. [ ] Assign production domains and SSL certificates.

11. **Post-Deploy Verification:**

12. [ ] Smoke test critical paths (signup, login, purchase flow).

13. [ ] Check Supabase and Vercel logs for errors.
14. [ ] Verify database migrations applied correctly.

15. [ ] Send test WhatsApp/email notifications to confirm external services.

16. **Monitoring:**

17. [ ] Enable alerts for high error rate.

18. [ ] Monitor resource usage (Supabase usage stats, Vercel analytics).
19. [ ] Have rollback steps on hand.

Costs & Risk Mitigation

Item	Free Tier	Paid (per)	Notes / Risks / Mitigation
Domain	–	₹900–1200/year	Standard domain cost.

Item	Free Tier	Paid (per)	Notes / Risks / Mitigation
Vercel	Hobby tier (free)	Pro $20/mo (teams)	Free covers small traffic. Upgrade if needed.
Supabase	Free: 2 projects, 500 MB DB, 50k MAUs【42†L255-L262】 5 GB bandwidth	Pro $25/org/mo (8 GB DB, 100k MAUs) 【42†L255-L262】	Free handles dev/testing. Scale to Pro at launch. Watch MAU and egress.
Resend (Email)	3,000 emails/mo 【50†L24-L32】	$10+/mo (extra)	3k/mo is ample for notifications.
Payment Gateway	0/month (transaction fees ~2%)	~2% per transaction	Razorpay/PhonePe have ~0-2% txn fee. Compare rates.
Meta WhatsApp API	Free sandbox (limit 1 number, templates)	Conversation-based pricing	Compliance risk if policy violated【23†L197-L205】. Only send templates.
Google Apps Script	~free (limited quota)	–	Good for small automations. No cost.
Other (Netlify, etc.)	Optional comparison	–	Vercel chosen for Next.js ease.

Notes: Free tiers are generous, but limits exist. E.g. Supabase pauses idle free projects after 7 days (data kept)【42†L255-L262】. Overshoots (MAU, storage) incur overage (flat rates). Set budget alerts. Ensure backups for DB changes. For Meta API, obtaining Business verification can be time-consuming – factor this into schedule.

Integration Options & Trade-offs

Service	Option A	Option B	Free Tier	Paid (start)	Trade-offs / Comments
Backend BaaS	**Supabase** (Postgres) 【42†L255-L262】	Firebase (Firestore)	Supabase: 2 projects, 500 MB DB, 50k MAUs 【42†L255-L262】 Firebase: Spark plan (limited reads/ writes, no auth MAU)	Supabase Pro $25/ org Firebase Blaze (pay as you go)	SQL (Relational) vs NoSQL. Supabase is open-source & predictable pricing; Firebase can spike on heavy I/O. Choose Supabase for SQL queries and RLS.

Service	Option A	Option B	Free Tier	Paid (start)	Trade-offs / Comments
Hosting	**Vercel**	Netlify	Both have free tier (small projects)	Vercel Pro $20/mo	Vercel: tight Next.js integration, Instant Rollback 【36†L206-L215】. Netlify is also easy but less optimized for Next.js.
Payments	**Razorpay**	PhonePe	No monthly fee, per-transaction fees (~2%)	–	Razorpay: widely used in SaaS, supports cards/UPI, subscriptions, good docs. PhonePe: UPI-focused, maybe cheaper UPI fees but limited to India and fewer features.
Email	**Resend**	SendGrid	Resend: 3k emails/mo free 【50†L24-L32】	SendGrid Essentials $14.95/mo	Resend has modern API and free 3k/mo. SendGrid is mature. For MVP, Resend suffices.
Messaging (WA)	**WhatsApp Cloud API** (Meta)	Twilio WhatsApp	Sandbox free, limited uses	Twilio Trial $15 credit	WhatsApp Cloud API: direct & free up to limits, but requires business approval and template compliance. Twilio: easier integration via one API for SMS/WhatsApp, but costs for WA messages.

Service	Option A	Option B	Free Tier	Paid (start)	Trade-offs / Comments
Storage	**Supabase Storage**	AWS S3 / Cloudflare R2	Supa: 5 GB free (Shared)	S3 (starts free 5GB)	Supa Storage: built-in RLS, CDN, image transforms【46†L101-L110】. R2 is cheap. S3 is classic but more setup. Supa is simplest for this stack.

Architecture & Deployment Diagrams

```
graph LR
  subgraph User App
    User[User Browser/App]
  end
  subgraph Frontend [Vercel (Next.js)]
    FE(Next.js UI/API)
  end
  subgraph SupabaseSvc [Supabase Services]
    Auth[(Auth)]
    DB[(Postgres DB)]
    Storage[(Storage)]
    EdgeFunc[Edge Functions]
  end
  subgraph Payment [Payment Gateway]
    PG(Razorpay / PhonePe)
  end
  subgraph Messaging [Meta (WhatsApp)]
    WA(WhatsApp API)
  end
  subgraph GoogleCloud [Google Apps Script]
    GAS(GAS Triggers)
  end

  User -->|HTTPS| FE
  FE -->|Supabase JS| Auth
  FE -->|CRUD| DB
  FE -->|File upload| Storage
  FE -->|initiate payment| PG
  PG -->|webhook POST| EdgeFunc
  EdgeFunc -->|DB write| DB
  EdgeFunc -->|send email| Resend[Resend Email API]
  EdgeFunc -->|send WhatsApp| WA
  DB -->|DB Hook| EdgeFunc
```

```
  GAS -->|REST API| DB
  GAS -->|send email| Gmail[Gmail]
  subgraph Monitoring
    VercelLogs[Logs/Monitoring]
    Sentry[Sentry]
  end
  FE --> VercelLogs
  EdgeFunc --> Sentry
```

```
graph TD
  GitHub[GitHub Repo] -->|push/PR| CI{GitHub Actions CI}
  CI -->|lint/test| BuildStep[Build Stage]
  BuildStep --> Preview[Preview Deploy (Vercel)]
  Preview -->|review & merge| Main[Main Branch]
  Main --> Production[Prod Deploy (Vercel)]
  Production --> Monitoring[Monitoring & Analytics]
```

SQL Schema Example

```sql
-- Users table (via Supabase Auth)
create table profiles (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users(id),
  email text unique not null,
  name text,
  created_at timestamptz default now()
);

-- Products & Plans
create table products (
  id serial primary key,
  name text not null,
  price integer not null
);

create table plans (
  id serial primary key,
  name text not null,
  features jsonb,
  price integer not null
);

-- Orders (purchase records)
create table orders (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users(id),
  plan_id int references plans(id),
  status text not null default 'pending',
```

```
  purchased_at timestamptz default now()
);

-- RLS policy example: only allow users to read their orders
alter table orders enable row level security;
create policy "Users select own orders"
  on orders for select
  using (user_id = auth.uid());
```

API/Webhook Example

```
# Example: Calling WhatsApp Cloud API (via Supabase Edge Function)
curl -X POST "https://graph.facebook.com/v17.0/$PHONE_NUMBER_ID/messages" \
  -H "Authorization: Bearer $WHATSAPP_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
        "messaging_product": "whatsapp",
        "to": "919999888777",
        "type": "template",
        "template": {
          "name": "order_shipped",
          "language": { "code": "en_US" },
          "components": [
            {"type":"body","parameters":[{"type":"text","text":"#12345"}]}
          ]
        }
      }'
```

```
# Example: Supabase Edge Function for Razorpay webhook
supabase functions invoke payment_webhook --body '{"payload": { ... }}'
```

References

- Supabase Docs: Auth methods and postgreSQL ecosystem【5†L133-L135】【5†L169-L177】; Edge Functions overview【1†L127-L134】; Storage features【46†L101-L110】; DB Webhooks【3†L162-L170】【3†L189-L197】.
- Vercel Docs: Environments & deployments【9†L283-L292】【9†L307-L312】; Instant Rollback【36†L206-L215】.
- Meta (WhatsApp) Policy: Messaging policy (24h window, templates, opt-in)【23†L178-L185】【23†L197-L205】.
- Google Apps Script: Triggers limitations (no auth services in simple triggers)【31†L1393-L1404】【31†L1420-L1424】.
- Accessibility: WCAG color contrast (min 4.5:1)【34†L209-L218】.
- Supabase Pricing (Free tier)【42†L255-L262】; Resend Email free tier【50†L24-L32】.
